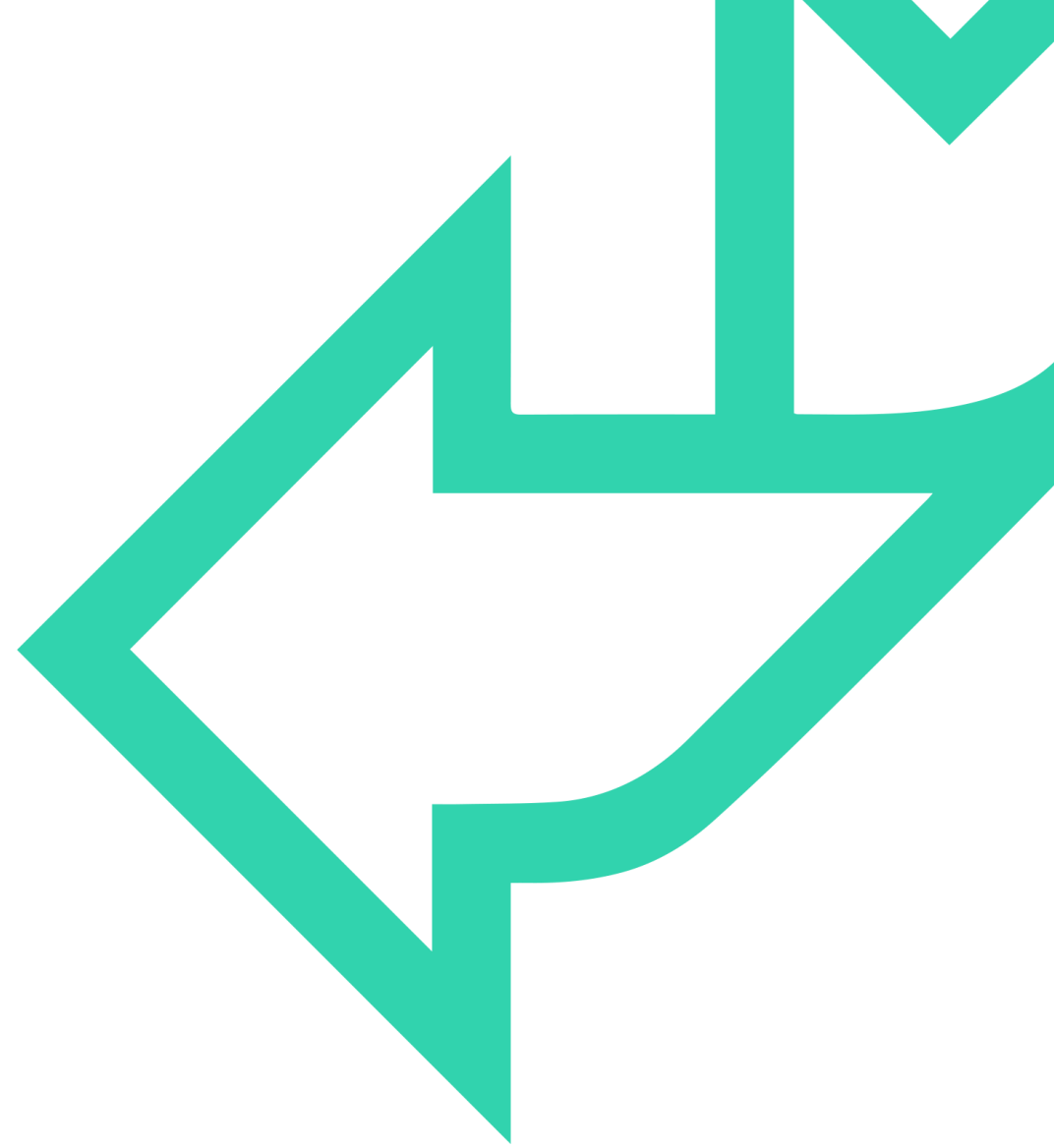




Jenkins Pipeline



QA Why Jenkins Pipelines Matter

- Traditional Jenkins jobs do not scale well
- UI-configured jobs are hard to track and maintain
- Changes are not version-controlled
- Pipelines treat delivery as code and integrate with Git

QA What is a Jenkinsfile

- A Jenkinsfile defines the pipeline for your project
- Stored in the same Git repository as your application
- Version controlled like source code
- Enables reproducible builds across machines and teams

```
pipeline {  
    agent any  
  
    stages {  
        stage('Build') {  
            steps {  
                echo 'Building app'  
            }  
        }  
  
        stage('Test') {  
            steps {  
                echo 'Running tests'  
            }  
        }  
  
        stage('Deploy') {  
            steps {  
                echo 'Deploying app'  
            }  
        }  
    }  
}
```

QA Basic Declarative Jenkinsfile Structure

- `pipeline { }` block defines the pipeline
- `agent` specifies where it runs (e.g., any node)
- `stages { }` contain logical steps
- `steps { }` perform actions like building or testing

```
pipeline {  
    agent any  
  
    stages {  
        stage('Build') {  
            steps {  
                echo 'Building app'  
            }  
        }  
  
        stage('Test') {  
            steps {  
                echo 'Running tests'  
            }  
        }  
  
        stage('Deploy') {  
            steps {  
                echo 'Deploying app'  
            }  
        }  
    }  
}
```

QA Declarative vs Scripted Pipelines

- Declarative: structured, easy to read and maintain
- Scripted: fully flexible, but more complex
- Declarative enforces a standard format, reducing mistakes

```
node {  
    if (env.BRANCH_NAME == 'main') {  
        stage('Build') {  
            echo 'Building'  
        }  
  
        stage('Test') {  
            echo 'Testing'  
        }  
    }  
}
```

QA Multistage Pipelines

- Break the pipeline into logical stages:
Build, Test, Package, Deploy
- Each stage has a single purpose
- Failures are easier to diagnose and fix

```
pipeline {  
    agent any  
  
    stages {  
        stage('Build') {  
            steps {  
                echo 'Building app'  
            }  
        }  
  
        stage('Test') {  
            steps {  
                echo 'Running tests'  
            }  
        }  
  
        stage('Deploy') {  
            steps {  
                echo 'Deploying app'  
            }  
        }  
    }  
}
```

QA Multistage Pipeline Example

- Checkout: Pull source code from Git
- Build: Compile or package code
- Test: Run unit, integration, or lint tests
- Package: Build artifacts or Docker images
- Deploy: Release to dev/test/production environment

```
pipeline {  
    agent any  
    stages {  
        stage('Checkout') {  
            steps {  
                checkout scm  
            }  
        }  
        stage('Build') {  
            steps {  
                sh 'mvn compile'  
            }  
        }  
        stage('Test') {  
            steps {  
                sh 'mvn test'  
            }  
        }  
        stage('Package') {  
            steps {  
                sh 'mvn package'  
            }  
        }  
        stage('Deploy') {  
            steps {  
                sh 'echo Deploy to environment'  
            }  
        }  
    }  
}
```

Parallel Stages

- **Run tasks simultaneously to save time**
- **Useful for independent work (unit tests, linting, or building multiple platforms)**
- **Reduces overall pipeline duration**

QA Parallel Stage Examples

- Unit tests and linting
- Integration tests on multiple environments
- Multi-platform builds (Windows/Linux/Mac)

```
stage('Quality Checks') {  
    parallel {  
        stage('Unit Tests') {  
            steps {  
                sh 'mvn test'  
            }  
        }  
        stage('Linting') {  
            steps {  
                sh 'mvn checkstyle:check'  
            }  
        }  
    }  
}
```

QA Conditional Stages

- Not all stages need to run every time
- Conditions allow flexibility based on branch, environment, or parameters
- Example: deploy only when building the main branch

```
stage('Deploy') {  
  when {  
    branch 'main'  
  }  
  steps {  
    echo 'Deploying to prod'  
  }  
}
```

QA Conditional Stages

- Not all stages need to run every time
- Conditions allow flexibility based on branch, environment, or parameters
- Example: run deploy.sh batch file only when on prod environment

```
stage('Deploy') {  
  environment {  
    ENV = 'prod'  
  }  
  when {  
    environment name: 'ENV', value: 'prod'  
  }  
  steps {  
    sh './deploy.sh'  
  }  
}
```

QA Manual Approvals

- Pause the pipeline at critical points
- Require human confirmation before proceeding
- Typically used before deploying to production

wait until someone approves

If the user clicks **Abort**, the pipeline will stop.



```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        echo 'Building'
      }
    }

    stage('Test') {
      steps {
        echo 'Testing'
      }
    }

    stage('Approve Production Deploy') {
      steps {
        input message: 'Deploy to production?'
      }
    }

    stage('Deploy') {
      steps {
        echo 'Deploying to production'
      }
    }
  }
}
```

QA Common Parameter Types

- String: free-form text input
- Choice: dropdown selection
- Boolean: true/false switch

```
pipeline {  
  agent any  
  parameters {  
    choice(  
      name: 'ENV',  
      choices: ['dev', 'test', 'prod'],  
      description: 'Target environment'  
    )  
    string(  
      name: 'VERSION',  
      defaultValue: '1.0.0',  
      description: 'Application version'  
    )  
    booleanParam(  
      name: 'RUN_EXTRA_TESTS',  
      defaultValue: false,  
      description: 'Enable extra tests'  
    )  
  }  
}
```

QA Parameters

- Accept input from users at runtime
- Examples: environment, version number
- Avoid duplicating jobs for different scenarios

```
pipeline {
  agent any
  parameters {
    choice(
      name: 'ENV',
      choices: ['dev', 'test', 'prod'],
      description: 'Target environment'
    )
    string(
      name: 'VERSION',
      defaultValue: '1.0.0',
      description: 'Application version'
    )
  }
}
```

```
stages {
  stage('Build') {
    steps {
      echo "Building version ${params.VERSION}"
    }
  }

  stage('Test') {
    steps {
      echo "Running tests in ${params.ENV}"
    }
  }

  stage('Deploy') {
    steps {
      script {
        if (params.ENV == 'prod') {
          echo "production deployment"
          // prod-specific commands here
        } else {
          echo "Deploying to ${params.ENV}"
          // Add dev/test commands here
        }
      }
    }
  }
}
```

QA Manual Approval + Conditional Example

- Deploy to prod only after manual confirmation
- Skip deploy in dev/test branches automatically
- Combine when condition with input step

```
pipeline {
  agent any

  stages {
    stage('Build') {
      steps {
        echo "Building..."
      }
    }

    stage('Approve Prod Deploy') {
      when {
        branch 'prod'
      }
      steps {
        script {
          input message: "Deploy to PROD?"
        }
      }
    }
  }
}
```

```
stage('Deploy') {
  when {
    anyOf {
      branch 'dev'
      branch 'test'
      branch 'prod'
    }
  }

  steps {
    script {
      if (env.BRANCH_NAME == 'prod') {
        echo "Deploying to PROD"
      } else {
        echo "Auto-deploying to ${env.BRANCH_NAME}
              (no approval needed)"
      }
    }
  }
}
```

Summary

- **Pipelines are code, not just UI jobs with structure improves reliability and clarity**
- **Can combine multistage, parallel, conditional, parameterised, and approval steps**
- **Use parallel testing to speed up feedback**
- **Use Conditional deployment for flexibility**
- **Use Manual approval for safety and Parameter-driven builds for reusability**